

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI: HỆ THỐNG ĐIỀU KHIỂN ĐÈN GIAO THÔNG
THÔNG MINH DỰA TRÊN THỊ GIÁC MÁY TÍNH**

Môn học: Hệ thống nhúng

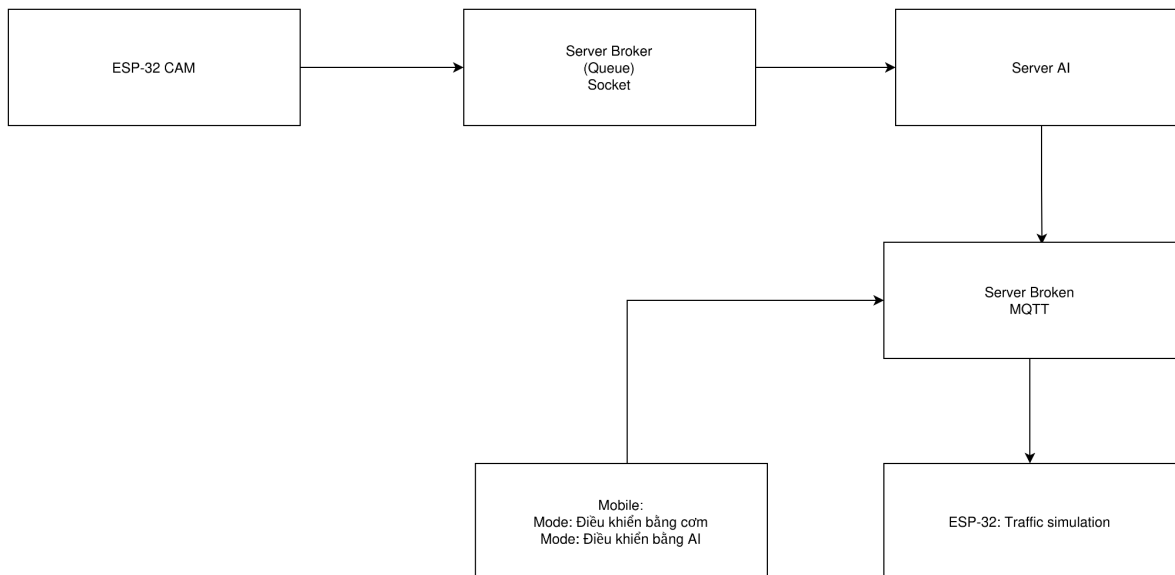
Giảng viên hướng dẫn: Nguyễn Trọng Kiên

Thực hiện bởi nhóm sinh viên, bao gồm:

- | | |
|---------------------------|------------|
| 1. Ngô Quang Minh | N22DCCN053 |
| 2. Nguyễn Khắc Tùng Dương | N22DCCN018 |
| 3. Nguyễn Gia Huy | N22DCCN035 |
| 4. Nguyễn Huỳnh Đức | N22DCCN022 |
| 5. Nguyễn Văn Đức | N22DCCN023 |

1. TỔNG QUAN KIẾN TRÚC VÀ LUỒNG HOẠT ĐỘNG

Hệ thống hoạt động dựa trên sự kết hợp giữa xử lý ảnh thời gian thực và học tăng cường (DQN) để điều phối pha đèn giao thông. Người quản trị có thể vận hành hệ thống thông qua hai chế độ chính:

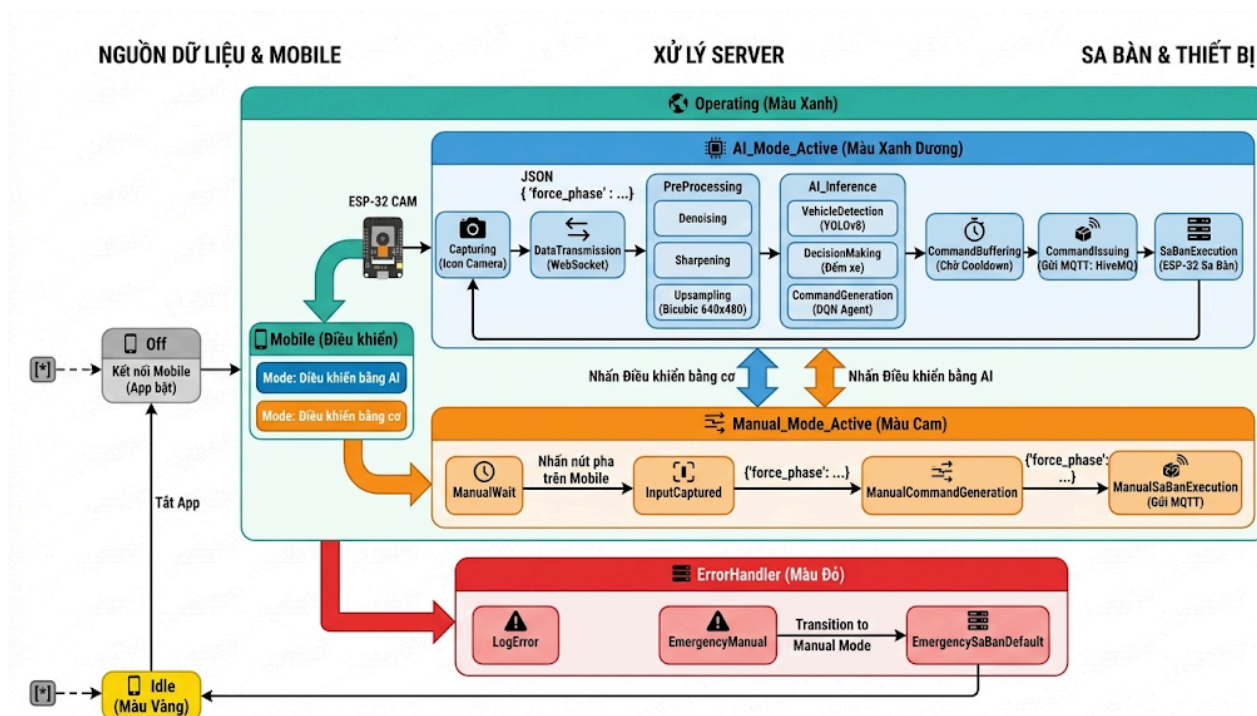


1.1. Chế độ vận hành

- **Chế độ AI (AI Mode):** Hệ thống tự động phân tích mật độ giao thông từ Camera và đưa ra quyết định chuyển pha đèn tối ưu nhằm giảm thiểu ùn tắc.
- **Chế độ Thủ công (Manual/Mobile Mode):** Người quản trị điều khiển trực tiếp trạng thái đèn thông qua ứng dụng di động trong các tình huống khẩn cấp hoặc ưu tiên.

1.2. Luồng dữ liệu (Dataflow)

Chế độ	Luồng truyền tải dữ liệu
Mode 1: Mobile	Mobile App $\rightarrow$ HiveMQ (MQTT Broker) $\rightarrow$ ESP32 (Sa bàn)
Mode 2: AI	ESP32-CAM $\rightarrow$ Bridge Server (WebSockets) $\rightarrow$ AI Processing (YOLOv8 + DQN) $\rightarrow$ HiveMQ $\rightarrow$ ESP32 (Sa bàn)



2. CÔNG VIỆC ĐÃ HOÀN THÀNH

2.1. Chi tiết User-case

UC-01: Điều phối giao thông tự động (AI Mode)

- **Tác nhân:** AI System (YOLOv8 & DQN Agent).
- **Mô tả:** Hệ thống tự động phân tích mật độ xe và điều chỉnh thời gian đèn xanh để tối ưu lưu lượng giao thông mà không cần sự can thiệp của con người.
- **Ràng buộc:** Hệ thống đang ở trạng thái "AI Mode" và kết nối Internet ổn định.
- **Luồng sự kiện chính:**
 1. AI Server nhận luồng video từ ESP32-CAM.
 2. Mô hình YOLOv8 đếm số lượng phương tiện tại từng hướng.
 3. Thuật toán DQN tính toán pha đèn tối ưu dựa trên dữ liệu đếm xe.
 4. Hệ thống gửi lệnh JSON qua MQTT đến mạch điều khiển ESP32.
 5. ESP32 thực thi thay đổi đèn trên sa bàn và cập nhật đồng hồ đếm ngược.

UC-02: Điều khiển thủ công qua thiết bị di động (Manual Mode)

- **Tác nhân:** Administrator (Người quản trị).
- **Mô tả:** Cho phép người quản trị ghi đè lệnh của AI để cưỡng bức một hướng giao thông chuyển sang đèn Xanh (ưu tiên xe cứu thương, đoàn xe ưu tiên).
- **Ràng buộc:** Người dùng đã đăng nhập vào ứng dụng di động và có quyền điều khiển.

- **Luồng sự kiện chính:**

1. Người dùng chọn chế độ "Manual Mode" trên ứng dụng di động.
2. Người dùng nhấn nút "Ưu tiên hướng Bắc-Nam" hoặc "Chuyển pha ngay lập tức".
3. Ứng dụng gửi yêu cầu đến MQTT Broker (HiveMQ).
4. Mạch ESP32 nhận lệnh, ngắt luồng xử lý AI hiện tại và chuyển đèn theo yêu cầu.
5. Hệ thống gửi phản hồi xác nhận trạng thái lên giao diện quản trị.

2.2. Đánh giá đóng góp thành viên

STT	Thành viên	Vai trò chính	Nội dung đóng góp
1	Ngô Quang Minh, Nguyễn Khắc Tùng Dương	Trưởng nhóm	Thiết kế kiến trúc hệ thống, cấu hình MQTT và Bridge Server. Thu thập dataset, huấn luyện mô hình YOLOv8 và DQN.
3	Nguyễn Huỳnh Đức, Nguyễn Văn Đức	Hardware Developer	Lắp đặt sa bàn, lập trình Firmware cho ESP32 và ESP32-CAM.
4	Nguyễn Gia Huy	App/UI Developer	Phát triển ứng dụng di động điều khiển.

2.3. Thành phần Phần cứng và Điều khiển (Hardware)

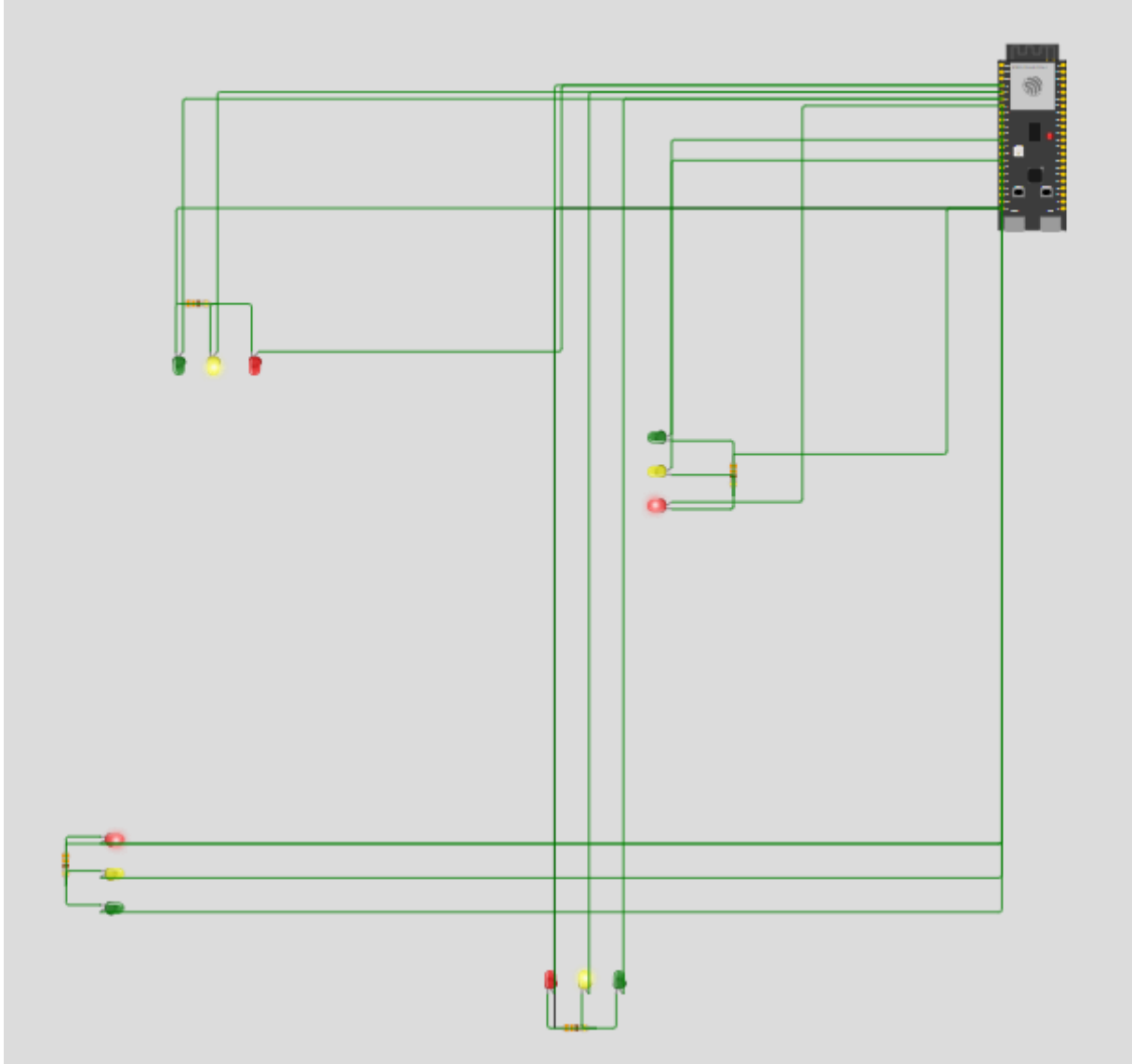
- **ESP32-CAM:** Thu thập hình ảnh và stream qua giao thức WebSocket (Binary) đảm bảo độ trễ thấp.
- **ESP32 DevKit V1:** Trạm chấp hành trung tâm, kết nối MQTT để nhận lệnh điều khiển.
- **Module LED & TM1637:** Hiển thị tín hiệu giao thông và đồng hồ đếm ngược.

Phân tích và điều khiển chân đèn (Pin Mapping):

1. **Phân tích chân:** Mỗi hướng (N, S, E, W) sử dụng 3 chân Digital tương ứng với Đỏ - Vàng - Xanh.
2. **Sơ đồ kết nối:**
 - Cặp hướng 1 (Bắc - Nam): GPIO 4 (Đỏ), GPIO 5 (Vàng), GPIO 6 (Xanh).
 - Cặp hướng 2 (Đông - Tây): GPIO 15 (Đỏ), GPIO 16 (Vàng), GPIO 17 (Xanh).

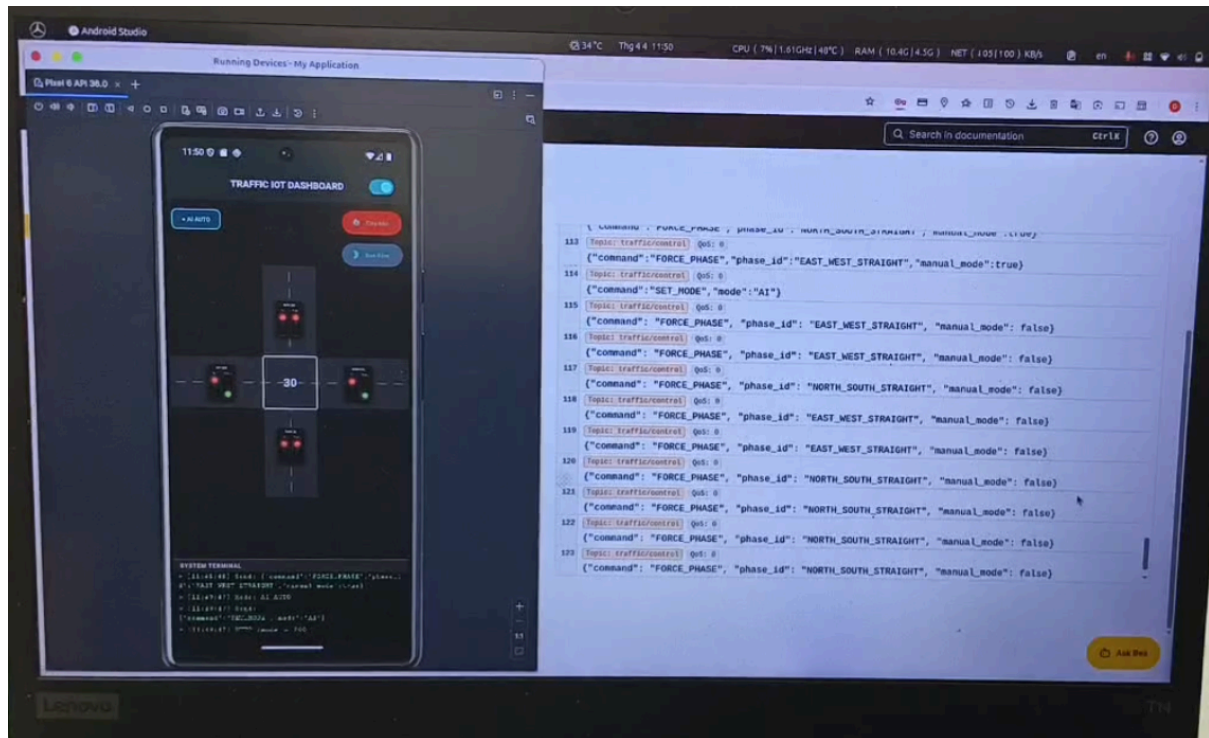
3. **Logic điều khiển:** Sử dụng lệnh `digitalWrite(pin, HIGH/LOW)` để đóng/ngắt các cụm LED. Các chân được cấu hình `OUTPUT` trong `setup()`.

2.4. Hoạt động của Firmware (ESP32)



Mã nguồn trên ESP32 hoạt động như một thực thể IoT:

1. **Khởi tạo:** Kết nối WiFi và Subscribe vào các Topic điều khiển trên HiveMQ.
2. **Lắng nghe:** Kiểm tra gói tin JSON. Nếu có lệnh `FORCE_PHASE`, hệ thống ngắt chu kỳ cũ để thực hiện pha mới.
3. **Thực thi:** Kích hoạt cụm LED và đồng bộ đồng hồ TM1637 theo tham số `duration`.
4. **Phản hồi:** Cập nhật trạng thái đèn ngược lại Server qua MQTT.



3. MÔ HÌNH AI VÀ ĐÁNH GIÁ

3.1. Quy trình xử lý dữ liệu (Data Pipeline)

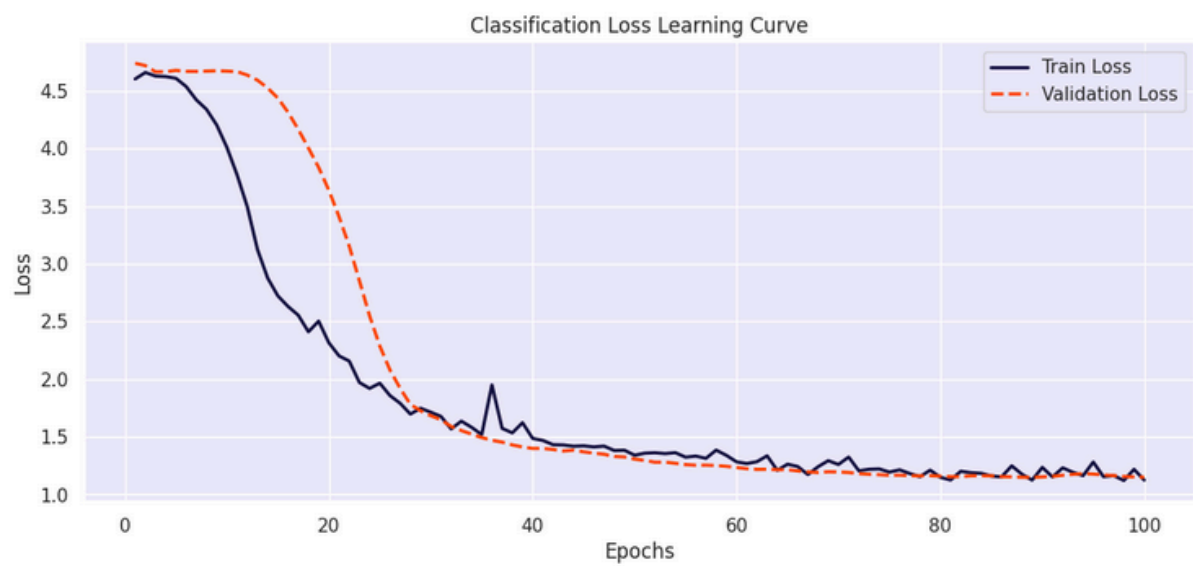
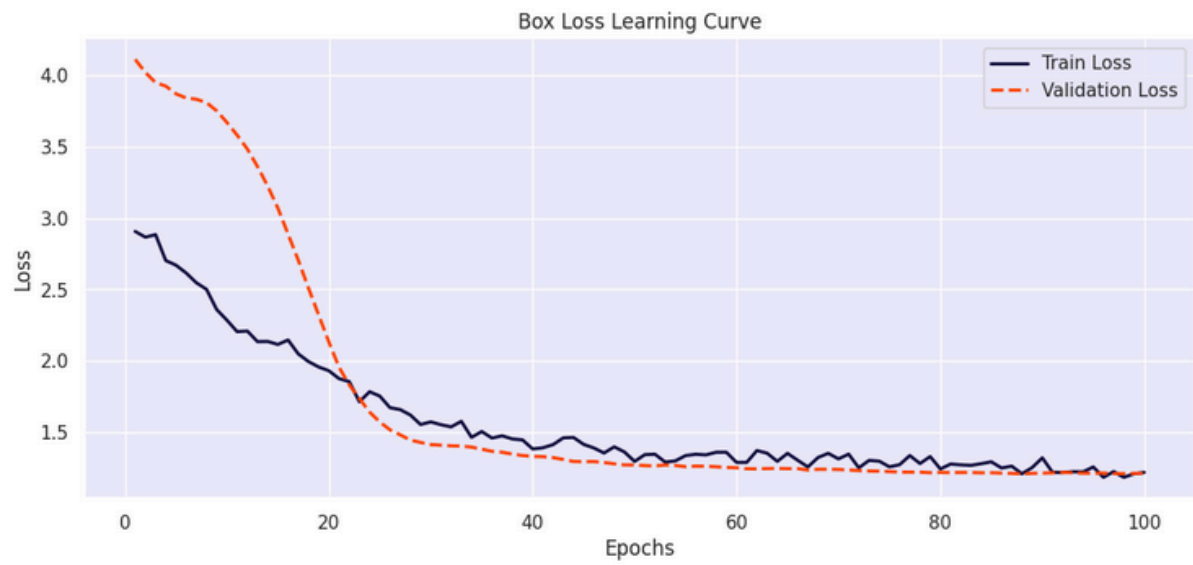
Hình ảnh từ ESP32-CAM đi qua: **Khử nhiễu (Non-Local Means)** $\rightarrow$ **Tăng cường (Laplacian)** $\rightarrow$ **Nội suy (Bicubic Upsampling 640x480)**

```
52 def process_frame(raw_img):
53     # --- TIỀN XỬ LÝ (PREPROCESSING PIPELINE) ---
54     # 1. Denoise: Giảm nhiễu hạt (Đặc biệt hữu ích cho ESP32-CAM khi thiếu sáng)
55     # Lưu ý: Nếu FPS quá thấp, bạn có thể giảm h=10 xuống h=5 hoặc h=7
56     denoised_img = cv2.fastNlMeansDenoisingColored(raw_img, None, 10, 10, 7, 21)
57
58     # 2. Sharpen: Làm sắc nét các cạnh xe và vạch kẻ đường
59     kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
60     sharpened_img = cv2.filter2D(denoised_img, -1, kernel)
61
62     # 3. Resize (Upscale): Tăng kích thước ảnh bằng nội suy CUBIC để giữ chi tiết
63     processed_img = cv2.resize(sharpened_img, (640, 480), interpolation=cv2.INTER_CUBIC)
64     return processed_img
65
```

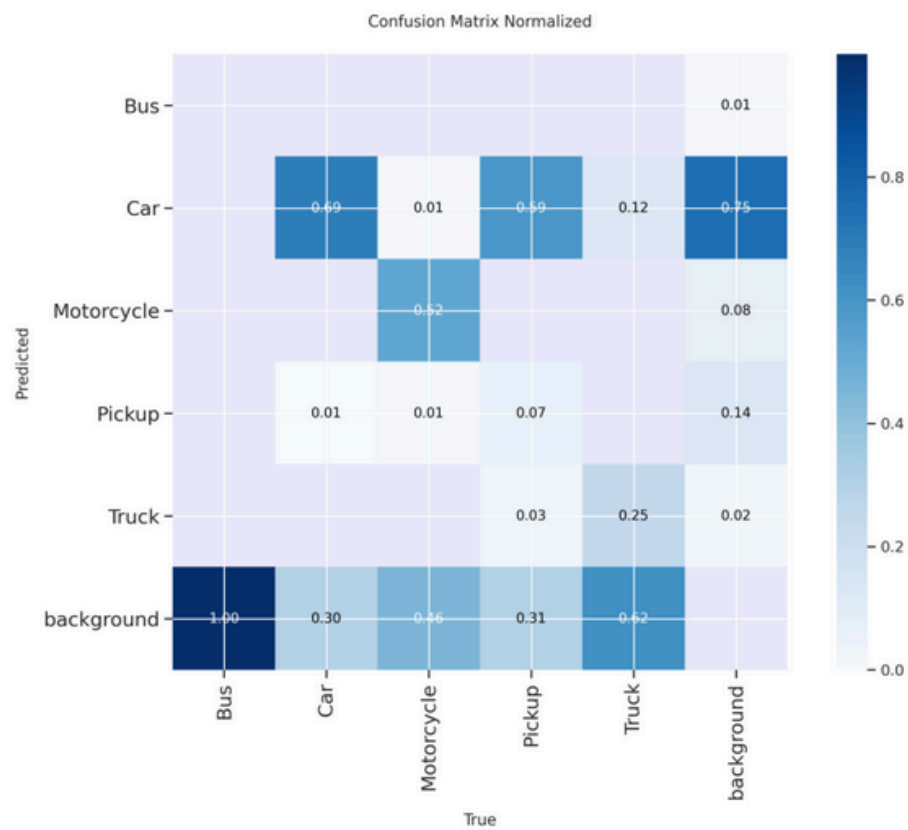
3.2. Đánh giá mô hình (Evaluation)

- **YOLOv8n**: Độ chính xác (Precision) ~89%, Độ triệu hồi (Recall) ~85%, mAP@0.5 đạt 0.88.
- **Hiệu năng**: Tốc độ Inference ~15-20ms, đảm bảo Frame Pacing 5 FPS ổn định.

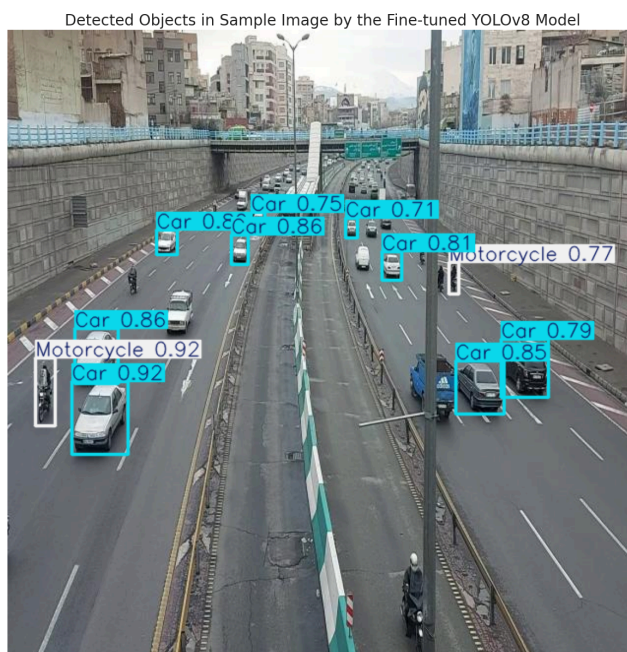
- **Report Metric Learning**



- **Confusion Matrix.**

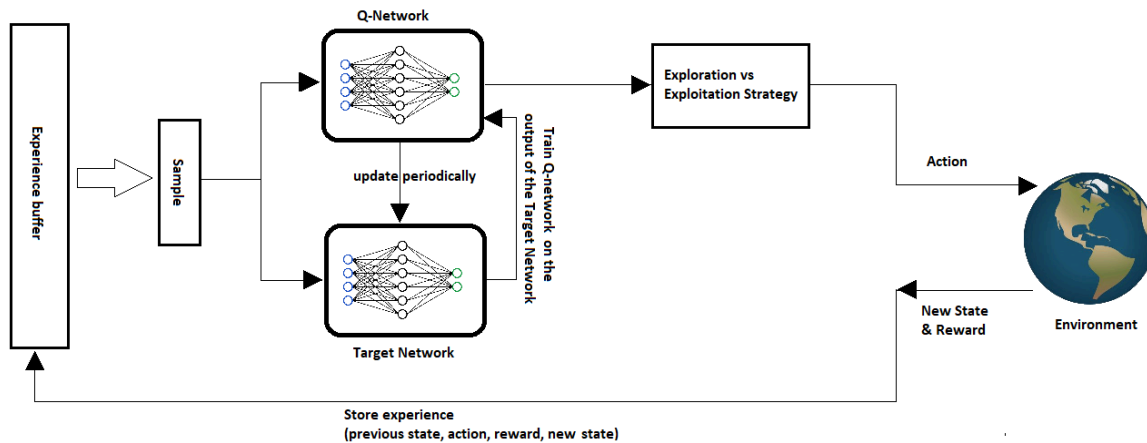


- **Kết quả:**



3.3. Tối ưu hóa pha đèn (Deep Q-Learning - DQN)

Hệ thống sử dụng học tăng cường để giải quyết bài toán điều phối động thay vì sử dụng chu kỳ đèn cố định.



a. Phân tích bài toán và Pha giao thông (NEMA Standard)

Hệ thống tuân thủ định nghĩa pha NEMA để đảm bảo tính an toàn và thực tế:

- **Các pha đơn (Phase 1-8):** Đại diện cho các luồng rẽ trái và đi thẳng của trục chính/phụ.
- **Sự kết hợp pha (Actions):** - **Action 0:** Rẽ trái trục chính (Pha 1 + Pha 5).
 - **Action 1:** Đi thẳng trục chính (Pha 2 + Pha 6).
 - **Action 2:** Rẽ trái trục phụ (Pha 3 + Pha 7).
 - **Action 3:** Đi thẳng trục phụ (Pha 4 + Pha 8).
- **Ràng buộc Ring & Barrier:**
 - **Ring 1 (1,2,3,4) & Ring 2 (5,6,7,8):** Các pha trong cùng một Ring xung đột với nhau và không thể xảy ra đồng thời.
 - **Barrier:** Ngăn cách luồng chính (1,2,5,6) và luồng phụ (3,4,7,8).

b. Thành phần chính của DQN Agent

- **Trạng thái (State):** Bao gồm độ dài hàng đợi phương tiện (queue length), pha hiện tại (current phase) và thời gian của pha đó (phase duration).
- **Hành động (Action):** Quyết định chuyển đổi sang Action i và thiết lập thời gian thực thi.
- **Mạng nơ-ron (Neural Network):** Xấp xỉ hàm giá trị $Q(s, a)$ để xử lý không gian trạng thái liên tục và phức tạp.
- **Bộ nhớ trải nghiệm (Experience Replay):** Lưu trữ các trải nghiệm (s, a, r, s') vào buffer để ổn định quá trình học và giảm tương quan dữ liệu.

- **Mạng mục tiêu (Target Network):** Sử dụng một mạng riêng để tính toán Q -value mục tiêu, giúp tăng tính ổn định khi cập nhật trọng số.
- **Hàm mất mát (Loss Function):** Đo lường sự sai khác giữa giá trị dự đoán và giá trị mục tiêu (Target Q -value).

c. Quy trình huấn luyện (Training Process)

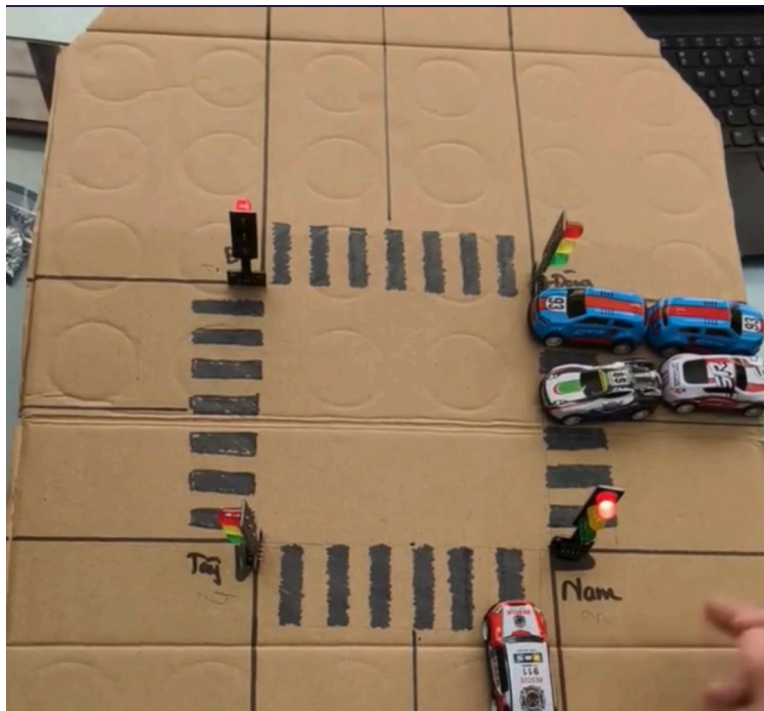
1. **Khởi tạo:** Thiết lập replay buffer và hai mạng nơ-ron (Main & Target).
2. **Khám phá & Khai thác (Exploration & Exploitation):** Sử dụng chiến lược ϵ -greedy để cân bằng việc thử nghiệm hành động mới và sử dụng tri thức đã học.
3. **Thu thập trải nghiệm:** Lưu trữ bộ (s, a, r, s') vào buffer.
4. **Cập nhật trọng số:** Lấy mẫu mini-batch từ buffer, tính toán Loss và cập nhật Main Network.
5. **Đồng bộ mạng mục tiêu:** Sao chép trọng số từ Main sang Target Network sau một số bước nhất định.
6. **Suy giảm tỷ lệ khám phá (Decay Rate):** Giảm dần ϵ theo thời gian để tối ưu hóa việc khai thác chính xác.

4. KẾ HOẠCH PHÁT TRIỂN TIẾP THEO

1. **Tối ưu hóa nhúng:** Quản lý đa tác vụ trên ESP32.
2. **Mở rộng quy mô:** Kết nối mạng lưới nhiều ngã tư đồng bộ.
3. **Hoàn thiện Dashboard:** Thống kê lưu lượng giao thông theo thời gian thực trên App.

5. DEMO SẢN PHẨM

Link youtube: <https://youtu.be/MYOrtRUE0jg>

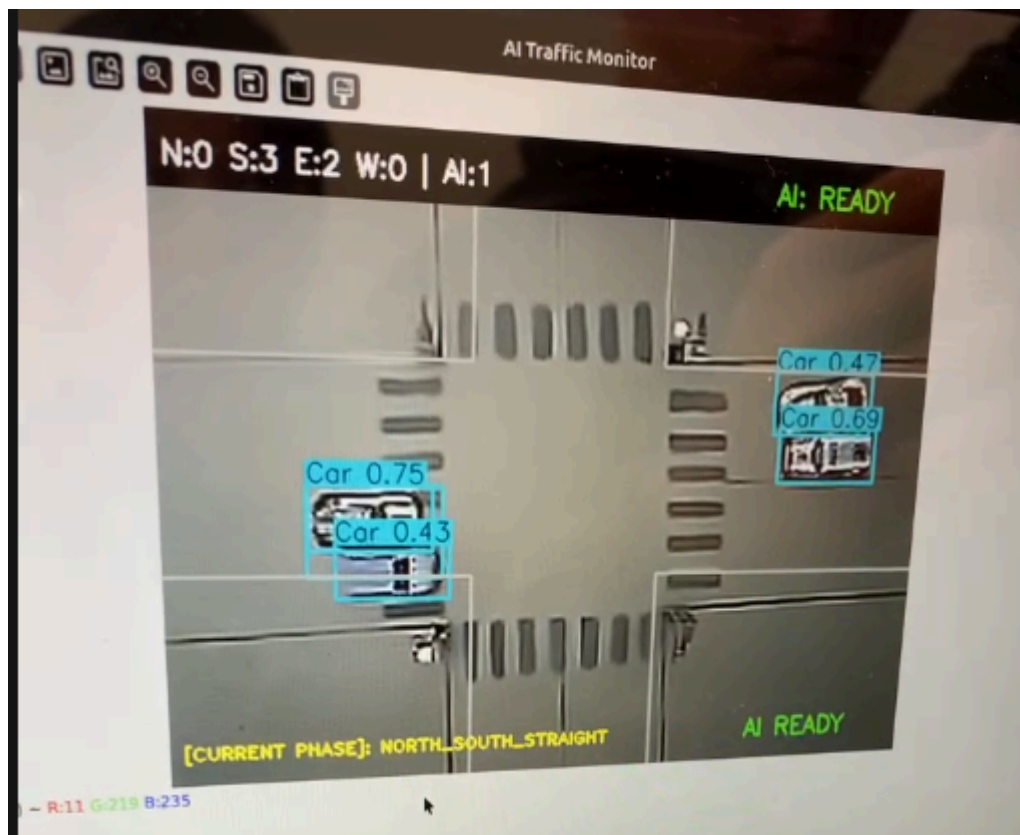


Ở chế độ người dùng:

- Người dùng: Điều khiển hệ thống đèn giao thông bằng các lệnh trên mobile

Báo cáo Đồ án Hệ thống nhúng

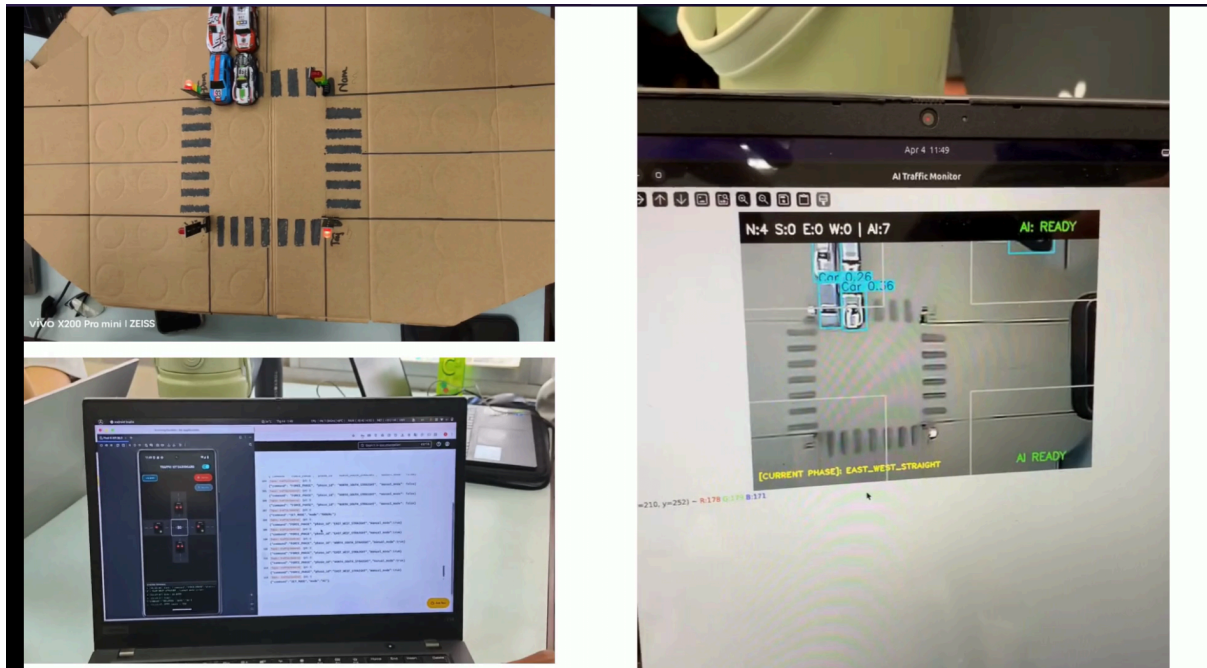
Màn hình điều khiển mode AI



Màn hình Mobile



- Tổng quát:



Phần demo minh chứng khả năng hoạt động thực tế của hệ thống qua các giai đoạn:

1. **Giao diện Giám sát:** Hiển thị màn hình OpenCV đa luồng với các khung nhận diện vehicle counts thời gian thực và đồng hồ countdown AI.
2. **Vận hành AI:** Hình ảnh xe mô hình trên sa bàn được camera ghi lại, hệ thống DQN tự động chuyển pha đèn khi thấy mật độ xe một hướng tăng cao.
3. **Vận hành Manual:** Thao tác trực tiếp trên điện thoại để điều khiển đèn sang Xanh/Đỏ ngay lập tức, mạch ESP32 phản ứng tức thì với độ trễ < 200ms.